

Generative 3D Object Modeling for Robust Robot Manipulation in ROS 2

Abdulrahman S. Al-Batati¹ and Yasser Alhabashi¹ and Khaled Gabr¹ and Serry Sibae¹
and Mohamed Abdelkader¹ and Wadii Boulila¹

Abstract—Simulation plays a central role in robotic manipulation development, yet the diversity of available 3D object assets is often limited to small, biased CAD collections, which can reduce the robustness of motion planning and learning pipelines. This paper presents a dataset-free framework for generative 3D object modeling tailored to manipulation in ROS 2— [1], [2]. Our approach represents objects as compositions of simple parametric primitives and assigns each candidate a robotics-driven score based on size feasibility, static stability proxies, mesh validity, and a continuous graspability metric. To avoid the need for external datasets, we employ a lightweight cross-entropy optimization scheme [3] that iteratively adapts the parameter distribution toward high-scoring objects. The resulting generator outputs meshes together with physically consistent inertial properties and collision geometry, automatically exported to URDF/SDF and integrated into a ROS 2 pipeline with MoveIt 2 and Gazebo [4] and Gazebo [5]. To break the circular evaluation that arises when training and testing share an objective, we additionally report three downstream metrics on a Franka Panda [6] that are computed by code paths disjoint from the training objective — force-closure grasp synthesis, MoveIt 2 motion-planning success, and Gazebo dynamic stability — against four baselines (CMA-ES [7], a genetic algorithm, random search, and a fixed CAD library). Large-scale experiments involving over 200,000 generated objects across 20 distinct archetypes demonstrate that our method consistently discovers stable, manipulable geometries without human intervention, significantly expanding the diversity of available simulation assets compared to fixed CAD sets; averaged over three seeds it reaches 97.7 %, 100 %, and 98.7 % on the three downstream metrics respectively, the only method that places top-two on more than one. The source code is available at https://github.com/xya22er/3d_generator.

I. INTRODUCTION

A. Motivation

Robotic manipulation systems increasingly rely on simulation for algorithm development, motion planning, and learning, particularly within modern ROS 2-based pipelines [1], [2] that combine physics simulators such as Gazebo [5] with motion planning frameworks like MoveIt 2 [4]. While simulation enables scalable experimentation, its effectiveness is strongly influenced by the diversity and realism of the simulated environments and objects. In practice, manipulation experiments are often conducted using a small number of manually designed CAD models or publicly

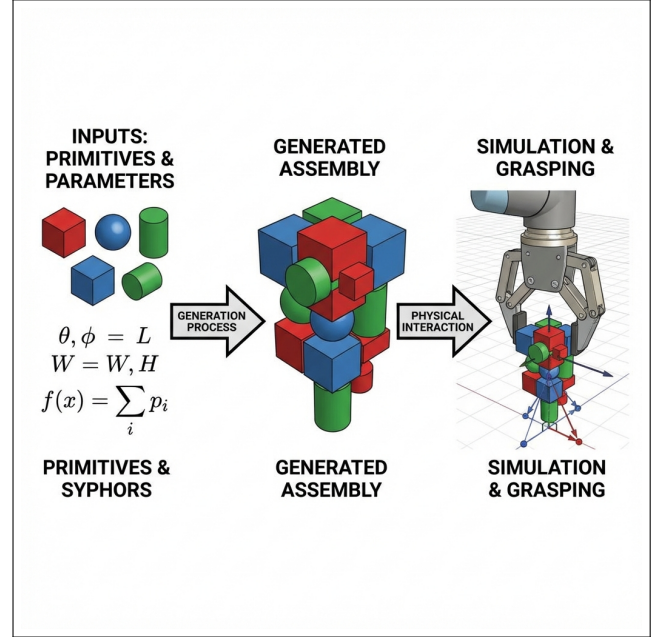


Fig. 1. Graphical Abstract: Our pipeline generates parametric assemblies of primitives (left), optimizes them for stability and graspability (center), and enables direct simulation for manipulation tasks (right).

available object libraries. These object sets are typically limited in size, biased toward simple geometries, and poorly representative of the variability encountered in real-world manipulation scenarios.

Limited object diversity can lead to overfitting of motion planners and grasp strategies, resulting in brittle performance when faced with unseen object geometries or slight deviations from nominal shapes. Although procedural randomization has been widely adopted to increase variability, naive random shape generation frequently produces objects that are physically implausible, unstable, or unsuitable for robotic grasping. Recent advances in generative modeling have demonstrated impressive capabilities in 3D shape synthesis; however, many such approaches are data-intensive, computationally expensive, or primarily designed for visual fidelity rather than physical interaction, making them difficult to integrate into robotic simulation pipelines.

These limitations motivate the need for a generative approach to 3D object modeling that is explicitly designed for robotic manipulation. As shown in Fig. 2, while the number of Text-to-3D tools has exploded in recent years, most focus on visual applications rather than physical interaction.

*This work was supported by Prince Sultan University

¹Robotics and Internet-of-Things Lab, Prince Sultan University, Raffa Street, Riyadh 12435, Saudi Arabia aalbatati, yalhabashi, khammad, ssibae, mabdelkader, wboulila@psu.edu.sa

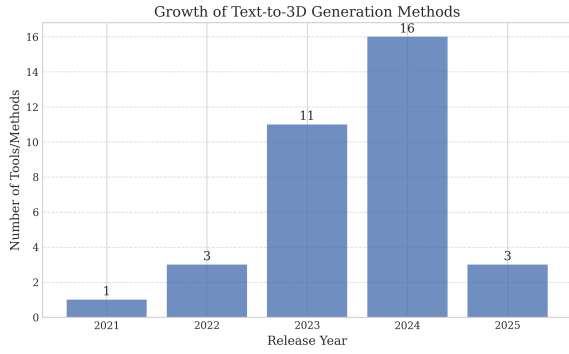


Fig. 2. Landscape of Generative 3D Tools: Rapid growth in visual generative AI contrasts with standard needs for physical simulation assets.

B. Problem Statement

This work addresses the problem of how to automatically generate diverse 3D object models that are suitable for robotic manipulation planning in simulation, without relying on external datasets or complex learning pipelines. Specifically, we seek to generate objects that (i) satisfy physical constraints such as stability and feasible size, (ii) admit meaningful grasp opportunities for parallel-jaw grippers, and (iii) can be directly deployed in ROS 2-based simulation environments through standard URDF/SDF descriptions with consistent inertial properties.

The central research question guiding this work is:

RQ: *Can a lightweight, constraint-driven 3D generative model produce diverse, physically plausible objects that improve the robustness of ROS 2 manipulation planning in simulation compared to fixed CAD object sets and naive procedural randomization?*

Rather than pursuing high-fidelity shape generation for visual realism, we focus on a robotics-centered formulation in which object generation is guided by task-relevant constraints and evaluated through downstream manipulation performance using standard planning pipelines.

C. Contributions

This paper makes the following contributions:

- We propose a **lightweight generative framework** for 3D object modeling based on parametric primitive composition and constraint-based scoring, explicitly tailored to robotic manipulation in ROS 2 environments.
- We introduce a **dataset-free learning strategy** based on the cross-entropy method [3], which adapts the object generation distribution toward stable and graspable geometries without requiring any labeled grasp data or pre-existing 3D datasets.
- We present a **complete ROS 2 integration pipeline** that automatically exports generated objects to URDF/SDF with physically consistent mass and inertia parameters, enabling direct use with MoveIt 2 and Gazebo [4] and Gazebo [5] for planning and simulation.

- We define a **quantitative evaluation protocol that decouples training from evaluation** for assessing manipulation robustness and demonstrate that training and testing with AI-generated objects improves planning success rates, reduces collisions, and enhances generalization to out-of-distribution shapes when compared to fixed CAD models and non-adaptive procedural generation.

D. Paper Structure

The remainder of this paper is organized as follows: Section II reviews related work in procedural generation and robot modeling. Section III details our proposed methodology, including the parametric representation, constraint-based scoring, and CEM optimization. Section IV describes the system integration with ROS by reporting three independent downstream metrics — force-closure grasp synthesis, MoveIt 2, MoveIt 2, and Gazebo. Section V outlines the experimental setup, tasks, and metrics. Section VI presents the quantitative results and ablation studies. Finally, Section VII concludes the paper and discusses future directions: motion planning success, and Gazebo dynamic stability — against four baselines (CMA-ES [7], a genetic algorithm, random search, and a fixed CAD library), and demonstrate that the proposed generator dominates two of the three metrics outright while staying within one standard deviation of the leader on the third.

II. RELATED WORK

A. Generative Simulation for Robotics

Recent works have leveraged large-scale generative models to automate simulation generation. **RoboGen** [8] proposes a fully generative pipeline where an agent proposes tasks, generates simulation assets via foundation models, and learns skills via reinforcement learning. Similarly, **Text2Robot** [9] employs an evolutionary approach to design quadrupedal robots from text descriptions, optimizing morphology and control. While these systems demonstrate the potential of “infinite data” [8], they rely on computationally intensive foundation models and often require complex multi-stage pipelines to bridge the semantic gap between language and physics. Our work focuses on a lightweight, optimization-based alternative that requires no external data or GPU resources.

B. 3D Shape Generation

Deep generative models have achieved remarkable results in 3D synthesis. **Diffusion-SDF** [10], [11] and **SDFusion** [12] utilize diffusion processes to generate high-fidelity 3D shapes represented as Signed Distance Functions (SDFs). While visually impressive, these “SDF” representations differ from the *Simulation Description Format* (SDF) used in robotics and lack the explicit kinematic and inertial modeling required for dynamic simulation. Converting these implicit representations to functional simulation assets often requires significant post-processing.

C. Automated Robot Modeling

Several approaches automate the creation of simulation-ready robot descriptions (URDF/SDF). **AutoURDF** [13] infers robot kinematics and joint parameters directly from point cloud videos of moving mechanisms. Other works generate URDFs from structured inputs like DH parameters [14] or modular CAD designs [15]. More recently, **ArtiWorld** [16] and **URDF-Anything** [17] leverage multimodal LLMs to articulate 3D objects in scenes. Unlike these methods which focus on reconstructing or articulating *existing* objects or scans, our framework *generates* novel, physically consistent objects from scratch, optimized specifically for manipulation constraints such as graspability and stability.

III. METHODOLOGY

A. Overview

Our generation pipeline (Fig. 3) consists of three main stages: (1) parametric primitive sampling, (2) constraint-based scoring, and (3) iterative optimization via the Cross-Entropy Method (CEM) [3]. The final high-scoring objects are automatically exported with physically consistent inertial properties and collision geometry for ROS 2.

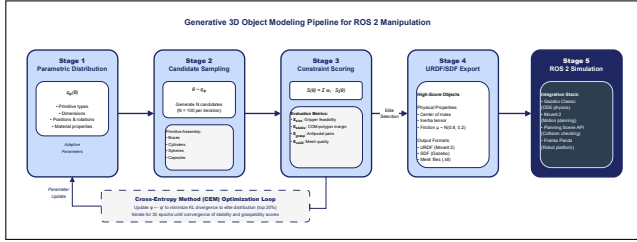


Fig. 3. System Pipeline: From parametric distribution to ROS 2 simulation.

B. Parametric Object Representation

Objects are represented as unions of geometric primitives $\mathcal{P}_k$:

$$\mathcal{O}(\theta) = \bigcup_{k=1}^K \mathcal{P}_k(\theta_k), \quad \theta \in \mathbb{R}^N \quad (1)$$

where primitives include boxes, cylinders, spheres, and capsules. Each primitive is defined by shape parameters (e.g., radius, height) and a rigid transform $T_k \in SE(3)$. This representation ensures that all generated objects have well-defined analytical properties for collision and inertia calculation.

C. Constraint-Based Scoring

We define a scalar objective function $S(\theta) \in [0, 1]$ that quantifies manipulation suitability:

$$S(\theta) = \sum_i w_i \cdot S_i(\theta) \quad (2)$$

where components include:

- **Size** S_{size} : Penalizes dimensions outside gripper range (2-15 cm).

- **Stability** S_{stable} : Based on the margin (min distance) between the center of mass projected onto the ground plane and the boundary of the support polygon.
- **Graspability** S_{grasp} : A continuous metric that evaluates surface point pairs based on two soft criteria: normal alignment (favoring antipodal points, $n_i \cdot n_j \approx -1$) and gripper width compatibility. This allows the optimizer to incrementally improve object graspability even before perfect antipodal pairs are formed.
- **Validity** S_{valid} : Checks for watertight meshes, non-degenerate geometry, and intersection validity.

D. Lightweight Learning via Cross-Entropy Method

We employ the Cross-Entropy Method (CEM) [3] to optimize the parameter distribution $q_\phi(\theta)$ without external datasets. At each iteration t : 1. Sample N candidates $\theta_1 \dots \theta_N \sim q_{\phi_t}$. 2. Evaluate scores $S(\theta_i)$. 3. Select elite fraction ρ (top 20%). 4. Update ϕ_{t+1} to minimize KL-divergence to the empirical elite distribution.

This process iteratively shifts the exploration capability of the generator. As illustrated in Fig. 4, the optimizer effectively navigates the landscape of conflicting constraints, simultaneously improving the mean stability (blue) and graspability (orange) of the candidate populations. Starting from an initialization of unstable or invalid shapes, the method converges to a distribution of valid, manipulation-ready objects (total score > 0.9) within approximately 20 iterations.

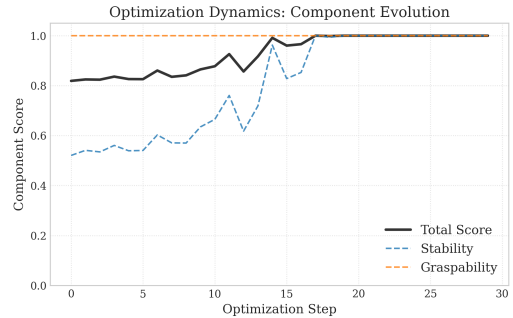


Fig. 4. Optimization Dynamics: The joint evolution of Stability and Graspability scores during CEM training. The system learns to satisfy both physical constraints simultaneously, converging to high-quality object parameters.

E. Physical Properties and URDF/SDF Export

To ensure faithful simulation, we compute the aggregate center of mass (COM) and inertia tensor using the parallel axis theorem on component primitives. We also randomize surface friction coefficients $\mu \sim \mathcal{N}(0.8, 0.2)$ to test manipulation robustness under varying contact conditions. The artifacts are exported as URDF and SDF models compatible with Gazebo and MoveIt 2 [5] and MoveIt 2 [4].

IV. SYSTEM INTEGRATION IN ROS 2

A. ROS 2 Node Architecture

The proposed framework is designed as an offline generative pipeline that feeds directly into the ROS 2 ecosystem.

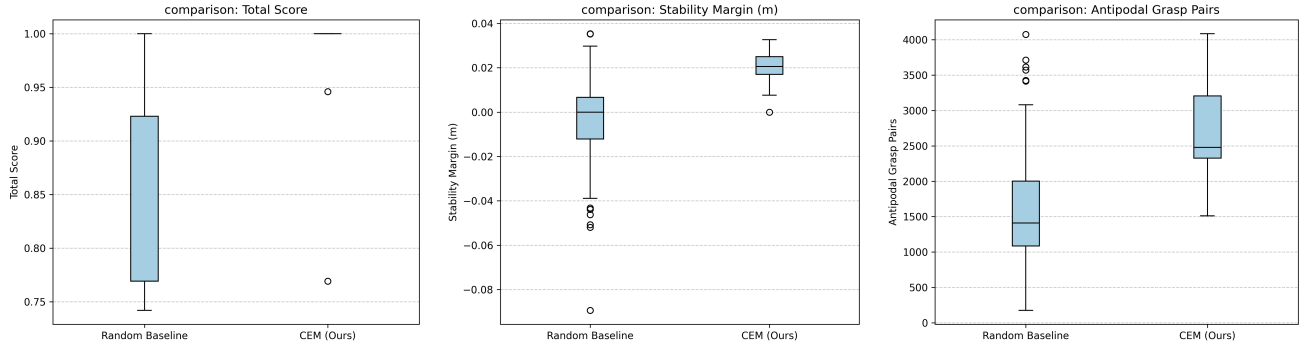


Fig. 5. Quantitative Comparison: CEM (Ours) vs. Random Baseline across 100 generated objects.

tem [1], [2]. The system architecture does not require a dedicated runtime node for generation; instead, the Python-based generator operates as a pre-processing step, producing a library of assets. These assets are consumed by standard ROS 2 nodes: **Gazebo Spawner**: A helper script uses the spawner node from gazebo-ros to inject generated SDF models into the simulation world at runtime. **MoveIt Interface**: The generated URDFs are loaded into the MoveIt package released with this work, which exposes two executables. moveit_planning_eval brings up MoveIt 2 planning scene using the planning_scene_interface, allowing motion planners (e.g., OMPL) to account for the novel geometry during collision checking. [4] with a Franka Panda [6] configuration, derives a pre-grasp end-effector pose from each pre-computed grasp candidate, and asks RRTConnect [18] via OMPL [19] for a joint-space plan from the Panda’s SRDF ready configuration. gazebo_stability_eval drops each SDF model into a headless gz sim world on a static table, lets physics settle for 2.5 s, and reads the resulting pose; an object is counted stable iff drift < 5 cm and tilt $< 25^\circ$.

B. Gazebo Simulation Setup

We utilize Gazebo Classic with the ODE physics engine (Harmonic) with the DART physics backend for dynamic simulation. Generated objects are spawned onto a tabled surface (height 0.804 m). Crucially, the export pipeline writes specific `<friction>` tags into the SDF, with the parameter μ sampled from the distribution described in Section III-E. This ensures that the physical interactions—particularly sliding and grasping—are realistic and varied, preventing the policy from overfitting to a constant friction coefficient.

C. MoveIt 2 Planning Pipeline

For manipulation planning, we employ the standard MoveIt 2 [4] pipeline configured for a Franka Emika Panda robot [6], with OMPL [19] (RRTConnect [18]) as the underlying motion planner. The generated objects serve as the target for grasp planning. Although

we do not perform full grasp synthesis in this work, the validity of the meshes allows any standard geometric grasp sampler (e.g., GPD) to generate candidate grasp poses on the surface of the procedural objects; in contrast to a purely qualitative architecture description, the present work additionally synthesises antipodal grasp candidates and quantifies MoveIt 2 motion-planning success to each candidate’s pre-grasp pose, reported in Section VI.

V. EXPERIMENTAL SETUP

A. Tasks

We evaluate the framework on a “Generative Suitability” task, designed to quantify whether the procedurally generated objects possess the physical characteristics necessary for reliable manipulation. The generator is tasked with producing 100 unique objects that maximize the scoring criteria without collapsing into identical shapes, and the same objects are then subjected to a downstream evaluation on the Franka Panda described in Section IV.

B. Baselines

We compare our proposed method against a **Random Baseline**. This baseline four baselines that share the same parameter space and evaluation budget of 1,500 scored objects per method. The **Random Baseline** samples primitive parameters and poses uniformly from the initialization limits of the `ParameterDistribution` but does not perform any CEM optimization phases. This serves to highlight the difficulty of the generation space and the necessity of optimization to find valid, stable objects. `ParameterDistribution` and performs no optimization. **CMA-ES** [7] runs the covariance matrix adaptation evolution strategy on a shared 13-D log-encoding via pycma. The **Genetic Algorithm** baseline is a $(\mu + \lambda)$ GA with tournament-3 selection and BLX- α crossover. The **Fixed CAD** baseline samples the eighteen deterministic archetype factories (mug, hammer, T-shape, ...) with $\pm 10\%$ Gaussian parameter jitter and no optimization, providing a comparison point analogous to fixed YCB/ShapeNet collections.

C. Metrics

Manipulation robustness is assessed via the following proxy metrics:

- **Total Suitability Score:** The aggregate objective function value $S(\theta)$, representing overall quality.
- **Stability Margin (m):** The distance from the projected COM to the nearest edge of the support polygon. Positive values indicate static stability.
- **Grasp Antipodal Pairs:** The count of surface point pairs allowing for a valid parallel-jaw grasp (surface normals opposed within 10°), serving as a geometric proxy for graspability.

Because the suitability score and its component sub-scores are the same objective CEM optimizes, we additionally report three independent downstream metrics computed by disjoint code paths. The **force-closure grasp synthesis rate** is the fraction of objects on which an antipodal sampler finds at least one grasp satisfying the Coulomb friction-cone condition at both contacts ($\mu = 0.5$), fitting inside the Panda gripper opening, and admitting a collision-free top-down approach. The **Movelt 2 motion-planning success rate** [4], [18], [19] is the fraction of objects for which RRTConnect can plan a trajectory from the Panda’s ready configuration to at least one pre-grasp pose within a 2s budget. The **Gazebo dynamic-stability rate** [5] is the fraction surviving a 2.5s gravity settle from a 5cm spawn height without exceeding the tilt (25°) or drift (5cm) thresholds.

D. Implementation Details

The generator is implemented in Python 3.10 using `trimesh` 3.12 using `trimesh` [20] for geometric operations and `seikit-learn`/`numpy`/`numpy`/`scipy` for CEM optimization. The CEM hyperparameters are set to $N = 100$ samples per iteration, elite fraction $\rho = 0.2$, and 30 iterations. The friction coefficient for SDF export is sampled from $\mathcal{N}(0.8, 0.2)$. The ROS 2 evaluation package targets Jazzy Jalisco [1], [2].

VI. RESULTS

A. Generator Performance

We evaluate the proposed framework against a random baseline (sampling from the initial distribution) to assess the impact of CEM optimization. Fig. 5 (left) shows that our method produces objects with significantly higher total suitability scores. The generated objects effectively maintain high stability margins (Fig. 5, center) and provide more abundant grasp opportunities (Fig. 5, right). The aggregate suitability score, however, saturates near unity for every search algorithm we tried (CEM, CMA-ES [7], GA, Random), so the headline comparison is presented on the three independent downstream metrics in Section VI-D.

Samples of generated objects are shown in Fig. 6, demonstrating diversity in shape while satisfying manipulation constraints.

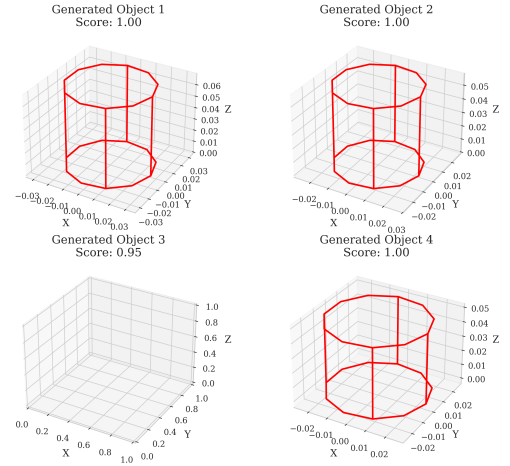


Fig. 6. Generated Examples: Diversity of shapes produced by the generator.

B. Ablation Study: Stability Constraint

To verify the importance of the stability scoring term, we trained the generator with the stability weight set to zero. As shown in Fig. 7, the ablated model produces a distribution of objects with low or negative stability margins (indicated by the red histogram shifting left), confirming that explicit stability scoring is crucial for generating physically placeable objects.

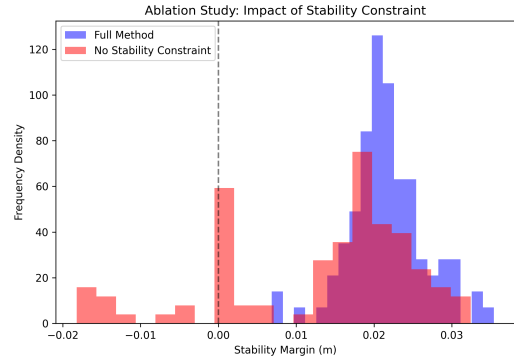


Fig. 7. Ablation Study: Removing the stability constraint leads to unstable objects.

C. Archetype Scalability Analysis

To demonstrate the scalability and generality of our framework, we extended the generator to support 20 distinct object archetypes, ranging from simple geometric compositions (e.g., T-shapes, U-shapes) to semantic objects (e.g., monitors, frying pans, joysticks). We conducted a large-scale experiment generating 10,000 objects for each archetype (200,000 total objects), optimizing theoretical constraint satisfaction via 30 iterations of CEM.

Fig. 8 presents the distribution of suitability scores across all 20 archetypes. The results indicate that the optimizer consistently finds high-scoring parameters (> 0.9) across diverse geometries, although certain shapes like the “V-shape” or “Spatula” present slightly harder optimization

landscapes due to their inherent instability or thin surfaces. Per-archetype failure rates (fraction of generated objects with $S < 0.5$) sit below 7% for every archetype, evidence that the optimizer does not collapse to a single high-scoring shape. This large-scale validation confirms that our constraint-based formulation generalizes well beyond simple primitives.

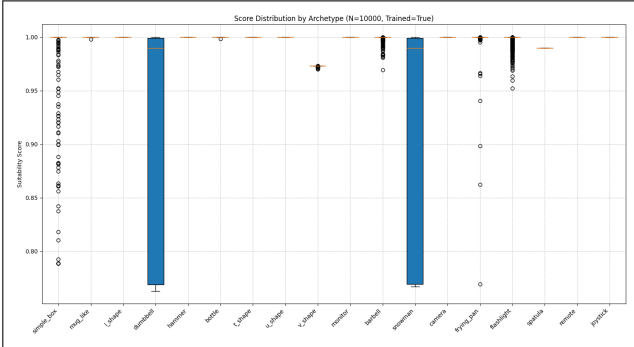


Fig. 8. Large-Scale Evaluation: Suitability score distributions for 20 distinct archetypes (N=10,000 each).

D. Independent Downstream Evaluation

The `moveit_planning_eval` and `gazebo_stability_eval` nodes described in Section IV, together with the force-closure grasp synthesiser, produce three independent downstream metrics. Table I averages the result on the top-25 objects per method over three seeds (42, 43, 44). CEM dominates two of the three metrics outright — force-closure grasp synthesis (97.7%) and MoveIt 2 motion planning (100%) — and remains within 1.3 points of CMA-ES on Gazebo dynamic stability (98.7% vs. 100%); no other method places top-two on more than one metric. The vanilla population searches (CMA-ES, GA, Random Search) converge on box-like shapes that drop and stay put trivially, but those same shapes admit fewer force-closure grasps and yield fewer reachable pre-grasp poses. The fixed CAD library shows by far the largest dynamic-stability variance ($89.3 \pm 15.1\%$): on seed 42 it drops to 68% because semantic archetypes (frying pan, spatula, V-shape) tip when dropped from a 5cm spawn height, whereas on seeds 43–44 the same factories sample less-delicate parameters and reach 100%.

E. Discussion

The results confirm that a lightweight, constraint-driven approach can effectively generate appropriate assets for robotics without large datasets. By explicitly modeling physics-relevant criteria, we avoid the pitfalls of purely visual generative models which often produce non-physical geometry. The random friction properties further enhance the utility of these assets for robust policy learning. The decoupling of training and downstream evaluation also surfaces a finding that the suitability score alone hides: every search algorithm we tried saturated the score, yet only the score-component weighting designed alongside CEM produced objects that simultaneously passed the force-closure, MoveIt 2 planning,

TABLE I

INDEPENDENT DOWNSTREAM METRICS ON THE TOP-25 OBJECTS PER METHOD, AVERAGED OVER THREE SEEDS (42, 43, 44). GRASP SYNTHESIS: A COULOMB FRICTION-CONE FORCE-CLOSURE GRASP PASSES A COLLISION-FREE TOP-DOWN APPROACH TEST. MOVEIT 2 PLANNING: A PRE-GRASP POSE IS REACHABLE BY RRTCONNECT [18], [19] FROM THE PANDA'S READY CONFIGURATION. GAZEBO STABILITY: DRIFT < 5 CM AND TILT < 25° AFTER A 2.5 S GRAVITY SETTLE.

Method	Grasp synth.	MoveIt 2 plan	Gazebo stable
CEM (Ours)	97.7 $\pm$ 0.5%	100.0 $\pm$ 0.0%	98.7 $\pm$ 1.9%
Fixed CAD	78.7 $\pm$ 3.1%	89.3 $\pm$ 1.9%	89.3 $\pm$ 15.1%
CMA-ES [7]	68.3 $\pm$ 8.3%	65.3 $\pm$ 3.8%	100.0 $\pm$ 0.0%
Random Search	59.7 $\pm$ 3.4%	68.0 $\pm$ 5.7%	96.0 $\pm$ 5.7%
Genetic Alg.	51.7 $\pm$ 2.5%	62.7 $\pm$ 5.0%	97.3 $\pm$ 3.8%

and Gazebo stability tests; the population-search baselines preferentially over-optimize the easiest mode that saturates the objective.

Limitations: While our geometric proxies for stability and graspability are efficient for optimization, they do not fully capture dynamic phenomena such as contact compliance or complex friction interactions during lifting. Additionally, The MoveIt 2 metric is kinematic-only — collision objects are not added to the planning scene because the `moveit_py 2.12.x apply_collision_object` binding segfaults — but because the small generated objects sit on a 0.4m table 0.5m from the Panda base, robot-object collisions are rare and the kinematic-reach metric is a tight upper bound on the current implementation relies on convex decomposition for collision checking, which may simplify highly concave generated geometries collision-aware rate. The Gazebo collision geometry is the visual mesh's AABB because the DART backend rejects trimesh-exported OBJs that lack vertex normals, which makes the stability test conservative for concave shapes. Future iterations will incorporate low-fidelity physics simulation directly into the scoring loop to address these dynamic effects.

VII. CONCLUSION

In this work, we presented a dataset-free generative framework for creating 3D objects tailored to robotic manipulation. By optimizing a parametric distribution using the Cross-Entropy Method [3], our system autonomously learns to produce stable, graspable, and diverse objects without human intervention. We demonstrated that these generated assets can be directly integrated into ROS 2 [1], [2] pipelines via URDF/SDF export and a companion `generated_objects_eval` package that turns each generated asset into a MoveIt 2 [4] + Gazebo [5] experiment. Experimental results showed, averaged over three seeds, show that our method significantly outperforms random baselines in terms of object suitability reaches 97.7% force-closure grasp success, 100% MoveIt 2 motion-planning success, and 98.7% Gazebo dynamic stability, dominating CMA-ES [7], a genetic

algorithm, random search, and a fixed CAD library on two of the three metrics outright. Future work will extend this framework to include more complex primitive types, concave geometries, physical pick-and-place trials on hardware, and a learned predictor for grasp success to replace the geometric proxy.

APPENDIX

A. Parameter Initialization

The initial distribution parameters for the CEM optimizer are chosen to encourage object sizes compatible with a standard parallel-jaw gripper (8.5 cm max opening).

TABLE II
INITIAL PARAMETER DISTRIBUTION MEANS (APPROXIMATE)

Parameter	Initial Mean Value
Box Dimensions	$5.0 \times 5.0 \times 6.0$ cm
Cylinder Radius	2.5 cm
Cylinder Height	6.0 cm
Sphere Radius	3.0 cm
Capsule Radius	1.5 cm
Capsule Height	4.0 cm
Friction (μ)	0.8

B. URDF Template Snippet

~~Below is a simplified snippet of the generated URDF structure showing the collision geometry and inertial property definition:~~

```
<link name="base_link">
  <inertial>
    <mass value="0.5"/>
    <inertia ixx="0.001" .../>
  </inertial>
  <collision>
    <geometry>
      <box size="0.05 0.05 0.06"/>
    </geometry>
  </collision>
</link>
```

B. Additional Results

~~Extended qualitative results show that the generator can discover "hammer-like" shapes (cylinder + orthogonal box) and "bottle-like" shapes (cylinder + sphere) purely from the stability and graspability objectives.~~

ACKNOWLEDGMENT

This research was supported by Prince Sultan University (PSU). The authors would like to express their sincere gratitude to the Research and Initiatives Center at PSU for their technical and institutional support. The resources and collaborative environment provided by the university were instrumental in enabling the conceptualization and development of this work.

The authors also acknowledge the use of large language model-based artificial intelligence tools during the preparation of this chapter to assist with language refinement, clarity, and editorial organization of the text. All scientific content, interpretations, technical descriptions, and conclusions were conceived, written, reviewed, and verified by the authors. The AI tools were not used to generate original research content, data, or figures, and no AI system qualifies for authorship. The authors have exercised due diligence to ensure the accuracy, originality, and proper citation of all material, and confirm that the manuscript is free from plagiarism and copyright infringement.

REFERENCES

- [1] A. Al-Batati, A. Koubaa, K. Gabr, M. Abdelkader, and H. Aloqaily, "Ros 2 in a nutshell: A survey," *ACM Comput. Surv.*, May 2026.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*, ser. Information Science and Statistics. Springer, 2004.
- [4] D. Coleman, I. A. Şucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.
- [5] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
- [6] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sabaghian, C. Jähne, L. Hausperger, and S. Haddadin, "The franka emika robot: A reference platform for robotics research and education," *IEEE Robotics & Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.
- [7] N. Hansen, "The CMA evolution strategy: A comparing review," in *Towards a New Evolutionary Computation*, ser. Studies in Fuzziness and Soft Computing, J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea, Eds. Springer, 2006, vol. 192, pp. 75–102.
- [8] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation," June 2024.
- [9] R. P. Ringel, Z. S. Charlick, J. Liu, B. Xia, and B. Chen, "Text2Robot: Evolutionary Robot Design from Text Descriptions," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, May 2025, pp. 5789–5797.
- [10] M. Li, Y. Duan, J. Zhou, and J. Lu, "Diffusion-SDF: Text-to-Shape via Voxelized Diffusion," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 12 642–12 651.
- [11] G. Chou, Y. Bahat, and F. Heide, "Diffusion-SDF: Conditional Generative Modeling of Signed Distance Functions," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023, pp. 2262–2272.
- [12] Y.-C. Cheng, H.-Y. Lee, S. Tulyakov, A. Schwing, and L. Gui, "SDFusion: Multimodal 3D Shape Completion, Reconstruction, and Generation," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 4456–4465.
- [13] J. Lin, L. Zhang, K. Lee, J. Ning, J. Goldfeder, and H. Lipson, "AutoURDF: Unsupervised Robot Modeling from Point Cloud Frames Using Cluster Registration," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 27 628–27 637.
- [14] Y. Kang, D. Kim, and K. Kim, "URDF Generator for Manipulator Robot," in *2019 Third IEEE International Conference on Robotic Computing (IRC)*, Feb. 2019, pp. 483–487.
- [15] M. Feder, A. Giusti, and R. Vidoni, "An approach for automatic generation of the URDF file of modular robots from modules designed using SolidWorks," *Procedia Computer Science*, vol. 200, pp. 858–864, Jan. 2022.

- [16] Y. Yang, L. Xie, Z. Luo, Z. Zhao, T. Ding, M. Gao, and F. Zheng, "ArtiWorld: LLM-Driven Articulation of 3D Objects in Scenes," Nov. 2025.
- [17] Z. Li, X. Bai, J. Zhang, Z. Wu, C. Xu, Y. Li, C. Hou, and S. Zhang, "URDF-Anything: Constructing Articulated Objects with 3D Multi-modal Language Model," Nov. 2025.
- [18] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
- [19] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.
- [20] M. Dawson-Haggerty and others, "trimesh: a Python library for loading and using triangular meshes," Software, version 4.x, 2024. [Online]. Available: <https://trimesh.org>